

AgentQO: Reliability-Aware Joint Logical and Physical Optimization for Multi-Agent LLM Workflows

Shusmoy Chowdhury

A research plan for serving multi-agent LLM applications on multi-GPU clusters

Multi-agent LLM applications are no longer single requests.

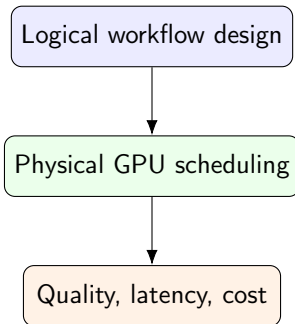
They now include:

- planner-executor chains;
- self-consistency and best-of- N branches;
- debate and refinement loops;
- verifiers, routers, memory, and tool calls.

These workflows can improve quality, but they also create large GPU cost, KV-cache pressure, and interference between concurrent workflows [1, 2, 3, 4, 5].

Problem

Current systems often separate two decisions.



The problem is that these decisions are coupled. A verifier, extra branch, or stronger model is useful only when its expected quality value is worth its current GPU cost.

Why This Is Hard

A runtime system must answer four questions at the same time.

- ① Should optional agent work exist now?
- ② Which model or fidelity level should run it?
- ③ Which GPU should execute it under current load?
- ④ Which KV state or speculative branch should be protected?

The answer changes as partial results arrive, tools return, queues grow, and KV state moves across GPUs.

Related Work: LLM Serving Foundations

Serving foundations provide the physical mechanisms that AgentQO would build on.

- vLLM improves KV memory efficiency through PagedAttention [6].
- Orca supports iteration-level scheduling for generative inference [7].
- Sarathi reduces prefill interference through chunked prefills [8].
- DistServe and Splitwise separate prefill and decode execution [9, 10].
- SGLang, CacheBlend, Mooncake, and KVLink improve reuse and KV-centric execution [11, 12, 13, 14].

These systems are strong, but they usually treat arriving work as work that must be served.

Related Work: Workflow-Aware Serving

Recent systems make serving more aware of programs and workflows.

- SAGA, Autellix, ThunderAgent, Pythia, and Maestro expose workflow or program structure to the serving layer [15, 16, 17, 18, 19].
- INFRAMIND adapts planning and reasoning decisions to live infrastructure state [20].
- GraphFlow treats dynamic agent workflows and state as a graph [21].
- LAMaS learns latency-aware workflow graphs and can prune redundant interactions [22].

This means the proposal should not claim that runtime workflow adaptation is missing. The gap must be narrower and more precise.

Three nearby lines of work are especially important.

- IconqSched predicts both candidate cost and the impact of a candidate on running work in database scheduling [23].
- Sherlock, MAST, and AgentForesight study selective verification, failure types, and online failure signals in agent workflows [24, 25, 26].
- KVFlow and TokenCake show that KV state should be managed with future-use information, not only with simple recency [27, 28].

AgentQO asks whether these ideas can be joined around one workflow-value signal.

The planned gap is not broad workflow awareness.

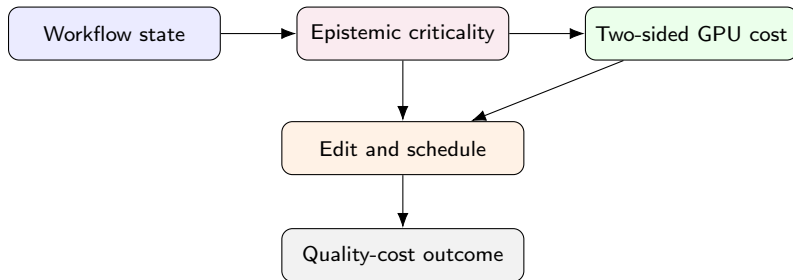
The gap is the combination of three decisions:

- ① estimate how much a node error can affect final task quality;
- ② estimate both candidate cost and interference caused to running calls;
- ③ use the same value signal for plan edits, placement, model fidelity, KV residency, and branch stopping.

I call the first signal **epistemic criticality**.

Main Idea

AgentQO treats logical workflow adaptation and physical scheduling as one runtime decision.



The goal is not to run larger workflows. The goal is to run only the work whose expected value justifies its physical cost.

I plan to organize the proposal around five questions.

- ① Can I predict two-sided interaction cost accurately enough for online scheduling?
- ② Can I estimate downstream epistemic criticality cheaply before a node finishes?
- ③ Does criticality-aware allocation improve quality per GPU-hour under a fixed workflow?
- ④ When does runtime workflow adaptation beat a fixed logical plan?
- ⑤ Can one value model guide several physical decisions without making the runtime unstable?

AgentIconq: Two-Sided Interaction Cost

For a candidate call a on GPU g , I plan to estimate two things.

$$\hat{L}_a(g \mid R_g, S_t)$$

Candidate completion cost under current GPU state.

$$\Delta_{a \rightarrow j} = \hat{L}_j(R_g \cup \{a\}, S_t) - \hat{L}_j(R_g, S_t)$$

Delay imposed on each running call j .

This matters because a call may look cheap for itself but still damage other workflows on the same GPU.

I define epistemic criticality as expected workflow-level quality loss caused by an error at a node.

$$EC_v = P(\text{err}_v \mid x_v) \cdot (\mathbb{E}[Q \mid v \text{ correct}] - \mathbb{E}[Q \mid v \text{ incorrect}])$$

This separates:

- local error probability;
- downstream consequence if that node is wrong.

A node can be moderately uncertain but still highly critical if many later decisions depend on it.

I will not start with arbitrary graph rewriting.

The first version will support local, controlled edits:

- increase or decrease best-of- N width;
- stop or extend a debate or refinement loop;
- add or skip a verifier;
- choose an approved model-fidelity level;
- cancel an optional speculative branch.

This keeps the online decision small and makes evaluation clearer.

At each event, AgentQO scores both a logical edit and a physical plan.

$$\max_{e, P} \quad \hat{Q}(W_t \oplus e) - \lambda_L \hat{L}(W_t \oplus e, P) - \lambda_C \hat{C}(W_t \oplus e, P) - \lambda_R \hat{R}(W_t \oplus e, P)$$

Here, e can also be no-op. The optimizer can keep the current workflow when extra work is not worth the cost.

KV Residency and Speculative Stopping

The same value signal can guide state and branch decisions.

For KV state owned by node v :

$$V_{KV}(b) = P(\text{reuse}_b) \cdot C_{\text{recompute}}(b) \cdot (1 + \kappa EC_v)$$

For an optional branch b :

$$I_b(t) = \frac{\mathbb{E}[\Delta Q_b \mid h_t]}{\mathbb{E}[C_b^{\text{remain}} \mid h_t]}$$

The system keeps a branch only while its expected marginal value remains competitive with other waiting work.

Planned System Stages

I plan to build the project in stages so each part can stand alone.

- ① reproduce and instrument a workflow-serving baseline;
- ② build the two-sided interaction predictor;
- ③ test epistemic criticality under a fixed plan;
- ④ add value-aware KV management and speculative stopping;
- ⑤ enable joint runtime plan edits and physical placement.

This staging reduces risk. If the full joint optimizer is too expensive, the earlier stages can still produce useful systems results.

I will evaluate in two layers.

Simulation. Extend a program-level agent-serving simulator with quality outcomes, optional branches, and logical edit events [29].

Hardware. Test on a multi-GPU system with different locality and KV movement conditions.

Workloads. Planner-executor, fan-out, supervisor-worker, self-consistency, debate, refinement, tool-heavy agents, and heterogeneous-model workflows.

The baseline set should match the capability being tested.

- request-level serving: vLLM, SGLang;
- workflow-aware serving: Autellix, SAGA, ThunderAgent, Pythia, Maestro;
- infrastructure-aware adaptation: INFRAMIND and LAMaS;
- workflow design: GPTSwarm and AFlow fixed graphs;
- reliability: Sherlock-style selective verification;
- KV management: LRU, KVFlow, TokenCake;
- value scheduling: Tempo.

System metrics

- median, P95, and P99 workflow completion time;
- throughput and SLO attainment;
- GPU utilization, HBM occupancy, KV hit rate, recomputation;
- wasted speculative GPU time.

Task metrics

- task quality;
- quality per GPU-hour;
- realized value and cost of each runtime edit.

I will stress the system under conditions where simple heuristics can break.

- high concurrency and bursty arrivals;
- wide workflows with many optional branches;
- long and variable tool gaps;
- heterogeneous model sizes and GPU types;
- high KV pressure and low reuse;
- noisy latency and criticality predictors;
- unseen workflow topology.

Risks and Fallbacks

- If criticality is too noisy, I will test simpler structural, disagreement, and verifier-based signals.
- If joint optimization overhead is too high, I will keep event-driven scoring and a smaller edit space.
- If fixed-plan criticality gives most of the benefit, I will treat that as the main result.
- If KV or branch-stopping performs strongly alone, either can become a focused contribution.

The plan is designed to produce useful evidence even if the full AgentQO design is only partly successful.

AgentQO is a plan to test one central idea:

Multi-agent serving should allocate GPU work based on both physical cost and downstream epistemic value.

If successful, the project would connect workflow reliability, interference-aware scheduling, KV-state management, and runtime plan adaptation under one systems framework.

References I



S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.



X. Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," ICLR, 2023.



S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," NeurIPS, 2023.



A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," NeurIPS, 2023.



Y. Du et al., "Improving Factuality and Reasoning in Language Models through Multiagent Debate," ICML, 2024.



W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP, 2023.



G.-I. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI, 2022.



A. Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," OSDI, 2024.



Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving," OSDI, 2024.



P. Patel et al., "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," ISCA, 2024.



L. Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," NeurIPS, 2024.

References II



J. Yao et al., “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion,” EuroSys, 2025.



R. Qin et al., “Mooncake: Trading More Storage for Less Computation, A KVCache-Centric Architecture for Serving LLM Chatbot,” FAST, 2025.



J. Yang et al., “KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse,” NeurIPS, 2025.



D. Guo et al., “SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters,” 2026.



M. Luo et al., “Autellix: An Efficient Serving Engine for LLM Agents as General Programs,” 2025.



H. Kang et al., “ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System,” 2026.



S. Yu et al., “Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving,” 2026.



J. Wang et al., “Maestro: Workload-Aware Cross-Cluster Scheduling for LLM-Based Multi-Agent Systems,” 2026.



A. Kabir et al., “INFRAMIND: Infrastructure-Aware Multi-Agent Orchestration,” 2026.



A. Li et al., “GraphFlow: A Graph-Based Workflow Management for Efficient LLM-Agent Serving,” 2026.



X. Shi et al., “Learning Latency-Aware Orchestration for Multi-Agent Systems,” 2026.

References III



Z. Wu et al., "Improving DBMS Scheduling Decisions with Accurate Performance Prediction on Concurrent Queries," PVLDB, 2025.



Y. Ro et al., "Sherlock: Reliable and Efficient Agentic Workflow Execution," 2025.



M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" 2025.



Anonymous, "AgentForesight: Online Auditing for Early Failure Prediction in Multi-Agent Systems," 2026.



Z. Pan et al., "KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows," NeurIPS, 2025.



Z. Bian et al., "TokenCake: A KV-Cache-Centric Serving Framework for LLM-Based Multi-Agent Applications," 2025.



R. H. Rajib et al., "AGENTSERVESIM: A Hardware-Aware Simulator for Multi-Turn LLM Agent Serving," 2026.